University of Petroleum and Energy Studies (UPES)

CL & NLP Assignment

LSTM Practice Questions

Name: Sagar Singh Thapa

Program: B.Tech CSE (Data Science)

Semester: VII

University: UPES, Dehradun

Computational Linguistics & Natural Language Processing

LSTM Equations Used

For the calculations, the sigmoid and hyperbolic tangent activation functions are

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

and

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}.$$

The standard equations of an LSTM cell used in this assignment are

$$f_t = \sigma\left(W_f x_t + U_f h_{t-1} + b_f\right),$$

$$i_t = \sigma\left(W_i x_t + U_i h_{t-1} + b_i\right),$$

$$\widetilde{C}_t = \tanh\left(W_c x_t + U_c h_{t-1} + b_c\right),$$

$$C_t = f_t C_{t-1} + i_t \widetilde{C}_t,$$

$$o_t = \sigma\left(W_o x_t + U_o h_{t-1} + b_o\right),$$

and

$$h_t = o_t \tanh(C_t).$$

Intermediate numerical values are rounded to approximately four decimal places.

Question 1: Forward Pass of a Single LSTM Cell

Given

$$x_t = 0.5, \qquad h_{t-1} = 0.2, \qquad C_{t-1} = 0.4.$$

1. Forget Gate

The pre-activation value for the forget gate is

$$z_f = 0.7(0.5) + 0.3(0.2) + 0.1.$$

Therefore,

$$z_f = 0.35 + 0.06 + 0.10 = 0.51.$$

Applying the sigmoid activation,

$$f_t = \sigma(0.51) = \frac{1}{1 + e^{-0.51}}.$$

Since

$$e^{-0.51} \approx 0.6005,$$

we obtain

$$f_t \approx \frac{1}{1.6005} \approx \boxed{0.6248}.$$

2. Input Gate

For the input gate,

$$z_i = 0.6(0.5) + 0.4(0.2).$$

Thus,

$$z_i = 0.30 + 0.08 = 0.38.$$

Hence,

$$i_t = \sigma(0.38) = \frac{1}{1 + e^{-0.38}}.$$

Using

$$e^{-0.38} \approx 0.6839,$$

we get

$$i_t \approx \boxed{0.5939}.$$

3. Candidate Cell State

The candidate cell-state pre-activation is

$$z_c = 0.9(0.5) + 0.2(0.2).$$

Therefore,

$$z_c = 0.45 + 0.04 = 0.49.$$

The candidate state is

$$\widetilde{C}_t = \tanh(0.49).$$

Using

$$\tanh(z) = \frac{e^{2z} - 1}{e^{2z} + 1},$$

we obtain

$$\widetilde{C}_t = \frac{e^{0.98} - 1}{e^{0.98} + 1}.$$

Since

$$e^{0.98} \approx 2.6645,$$

$$\widetilde{C}_t = \frac{2.6645 - 1}{2.6645 + 1} = \frac{1.6645}{3.6645} \approx \boxed{0.4542}.$$

4. Updated Cell State

The new cell state is

$$C_t = f_t C_{t-1} + i_t \widetilde{C}_t.$$

Substituting the values,

$$C_t = (0.6248)(0.4) + (0.5939)(0.4542).$$

Therefore,

$$C_t \approx 0.2499 + 0.2697 \approx \boxed{0.5196}.$$

5. Output Gate

For the output gate,

$$z_o = 0.5(0.5) + 0.5(0.2) + 0.1.$$

Thus,

$$z_o = 0.25 + 0.10 + 0.10 = 0.45.$$

Applying the sigmoid function,

$$o_t = \sigma(0.45) = \frac{1}{1 + e^{-0.45}}.$$

Since

$$e^{-0.45} \approx 0.6376,$$

$$o_t \approx \boxed{0.6106}.$$

6. Hidden State

Finally,

$$h_t = o_t \tanh(C_t).$$

First,

$$\tanh(0.5196) = \frac{e^{1.0392} - 1}{e^{1.0392} + 1} \approx 0.4774.$$

Hence,

$$h_t = (0.6106)(0.4774) \approx \boxed{0.2915}.$$

Final Results

$$\boxed{\begin{aligned} f_t &= 0.6248, \\ i_t &= 0.5939, \\ \widetilde{C}_t &= 0.4542, \\ C_t &= 0.5196, \\ o_t &= 0.6106, \\ h_t &= 0.2915. \end{aligned}}$$

The forget gate preserves approximately 62.48% of the previous cell-state contribution, while the input gate controls how strongly the new candidate information is incorporated. As a result, the cell state increases from 0.4 to approximately 0.5196.

Question 2: Number of Parameters in an LSTM

Let

$$m = 10$$

be the input size and

$$n = 32$$

be the hidden-state size.

An LSTM contains four sets of parameters corresponding to the forget, input, candidate, and output components.

For each component:

$$W \in \mathbb{R}^{n \times m}, \qquad U \in \mathbb{R}^{n \times n}.$$

(a) One Bias Vector per Gate

The number of parameters for one gate is

$$nm + n^2 + n.$$

Equivalently,

$$n(m + n) + n.$$

For all four components,

$$N = 4[n(m+n)+n].$$

Substituting $m = 10$ and $n = 32$,

$$N = 4[32(10+32)+32].$$

$$= 4[32(42)+32]$$

$$= 4[1344+32]$$

$$= 4(1376).$$

Therefore,

$$\boxed{N = 5504}.$$

Hence, the LSTM has $\boxed{5504}$ trainable parameters when one bias vector is used for each gate.

(b) PyTorch-Style Two Bias Vectors

PyTorch's LSTM implementation maintains two bias vectors associated with the input-hidden and hidden-hidden transformations.

Therefore,

$$N_{\text{PyTorch}} = 4[n(m+n)+2n].$$

Substituting the values,

$$N_{\text{PyTorch}} = 4[32(42)+64]$$

$$= 4[1344+64]$$

$$= 4(1408).$$

Thus,

$$\boxed{N_{\text{PyTorch}} = 5632}.$$

The difference is

$$5632 - 5504 = \boxed{128}.$$

Therefore, the two-bias formulation contains 128 additional bias parameters.

Question 3: Vanishing Gradient Through the Cell State

Assuming a constant forget-gate value f, the gradient contribution across k time steps can be approximated by

$$G_k \approx f^k.$$

(a) For $f = 0.9$

For $k = 10$,

$$G_{10} = 0.9^{10}.$$

Therefore,

$$\boxed{G_{10} \approx 0.3487}.$$

For $k = 50$,

$$G_{50} = 0.9^{50}.$$

Hence,

$$\boxed{G_{50} \approx 0.0052}.$$

Thus, after 50 steps, only about 0.52% of the original gradient magnitude remains under this simplified approximation.

(b) Comparison for $k = 50$

For $f = 0.5$,

$$G_{50} = 0.5^{50} \approx \boxed{8.88 \times 10^{-16}}.$$

This is effectively zero for practical learning.

For $f = 0.99$,

$$G_{50} = 0.99^{50} \approx \boxed{0.6050}.$$

Thus, approximately 60.5% of the gradient magnitude remains.

(c) Interpretation

These calculations show why the forget gate is important for learning long-term dependencies.

If $f = 0.5$, the gradient becomes almost zero after 50 time steps. Even at $f = 0.9$, only about 0.5% remains. In contrast, when the forget gate stays close to one, such as $f = 0.99$, a substantial portion of the gradient can propagate through many time steps.

A conventional RNN repeatedly multiplies gradients by recurrent weights and activation derivatives. These repeated multiplications can cause the gradient to decrease rapidly.

The LSTM cell-state update,

$$C_t = f_t C_{t-1} + i_t \widetilde{C}_t,$$

provides a more direct additive path for information. In the simplified cell-state path,

$$\frac{\partial C_t}{\partial C_{t-1}} \approx f_t.$$

Because the forget gate is learned, an LSTM can make f_t close to one when information needs to be preserved. This helps reduce the vanishing-gradient problem and makes long-term dependencies easier to learn.

Question 4: Electricity Load Forecasting

Given:

- Lookback period = 168 hours,

- Forecast horizon = 24 hours,
- Number of features = 3,
- Dataset duration = 3 years,
- Batch size = 64.

(a) Total Number of Hourly Records

Assuming 365 days per year,

$$N = 3 \times 365 \times 24.$$

Therefore,

$$\boxed{N = 26,280}.$$

So the dataset contains 26,280 hourly observations.

(b) Number of Sliding Windows

Each training example requires

$$168 + 24 = 192$$

consecutive hourly records.

For stride 1, the number of windows is

$$N_{\text{windows}} = 26,280 - 192 + 1.$$

Therefore,

$$\boxed{N_{\text{windows}} = 26,089}.$$

(c) Input and Output Shapes

Using the batch-first convention, the input tensor has the form

$$(\text{batch}, \text{time steps}, \text{features}).$$

Therefore,

$$\boxed{\text{Input shape} = (64, 168, 3)}.$$

If the model predicts 24 load values directly, the target/output shape is

$$\boxed{\text{Output shape} = (64, 24)}.$$

If the output explicitly preserves a single feature dimension, the shape can instead be written as

$$\boxed{(64, 24, 1)}.$$

(d) Number of Training Mini-Batches

Using a chronological 70% training split,

$$N_{\text{train}} = 0.70(26,280) = 18,396.$$

Only windows that lie completely inside the training portion should be used.

Thus,

$$N_{\text{train windows}} = 18,396 - 192 + 1.$$

Therefore,

$$\boxed{N_{\text{train windows}} = 18,205}.$$

With a batch size of 64,

$$\frac{18,205}{64} \approx 284.45.$$

Thus, if the final incomplete batch is retained,

$$\boxed{285 \text{ mini-batches}}.$$

There are 284 complete batches because

$$284 \times 64 = 18,176.$$

The remaining number of windows is

$$18,205 - 18,176 = 29.$$

Hence, the final mini-batch contains

$$\boxed{29 \text{ samples}}.$$

(e) LSTM Parameters

For

$$m = 3, \qquad n = 64,$$

and one bias vector per gate,

$$N_{\text{LSTM}} = 4[n(m+n) + n].$$

Substituting,

$$N_{\text{LSTM}} = 4[64(3+64) + 64].$$

$$= 4[64(67) + 64]$$

$$= 4[4288 + 64]$$

$$= 4(4352).$$

Therefore,

$$\boxed{N_{\text{LSTM}} = 17,408}.$$

Thus, the LSTM layer contains 17,408 trainable parameters.

If a fully connected output layer maps the 64-dimensional hidden state to 24 forecast values, it would contain

$$64(24) + 24 = 1536 + 24 = \boxed{1560}$$

additional parameters. These parameters are separate from the LSTM layer itself.

Question 5: Predictive Maintenance

(a) Min–Max Normalisation

For a sensor value $x = 95$, minimum $x_{\min} = 20$, and maximum $x_{\max} = 120$, min–max normalisation is

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Therefore,

$$x' = \frac{95 - 20}{120 - 20} = \frac{75}{100}.$$

Hence,

$$\boxed{x' = 0.75}.$$

(b) Classification Using the Threshold

Suppose the model predicts a failure probability of

$$P(\text{failure}) = 0.83$$

and the classification threshold is

$$T = 0.5.$$

Since

$$0.83 > 0.5,$$

the model classifies the pump as

$$\boxed{\text{Predicted Failure}}.$$

Therefore, according to the model, the pump is expected to fail in the next cycle.

(c) Confusion Matrix and Evaluation Metrics

Given:

$$\text{Total samples} = 1000,$$

$$\text{Actual failures} = 40,$$

$$\text{Predicted/flagged failures} = 45,$$

and 30 of the flagged cases are correct.

Therefore,

$$TP = 30.$$

The number of false positives is

$$FP = 45 - 30 = 15.$$

The number of false negatives is

$$FN = 40 - 30 = 10.$$

Finally,

$$TN = 1000 - TP - FP - FN.$$

Thus,

$$TN = 1000 - 30 - 15 - 10 = 945.$$

The confusion matrix is therefore

	Predicted Failure	**Predicted OK**
Actual Failure	30	10
Actual OK	15	945

Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{30}{30 + 15}.$$

Therefore,

$$\boxed{\text{Precision} = 0.6667}.$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{30}{30 + 10}.$$

Thus,

$$\boxed{\text{Recall} = 0.7500}.$$

F1 Score

The F1 score is

$$F_1 = 2\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Substituting,

$$F_1 = 2\frac{(0.6667)(0.75)}{0.6667 + 0.75}.$$

Therefore,

$$\boxed{F_1 \approx 0.7059}.$$

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{1000}.$$

Thus,

$$\text{Accuracy} = \frac{30 + 945}{1000} = \frac{975}{1000}.$$

Hence,

$$\boxed{\text{Accuracy} = 0.975 = 97.5\%}.$$

(d) Which Metric Should Be Prioritised?

For a predictive-maintenance system, recall is particularly important when failing to detect an actual equipment failure is more costly than performing an unnecessary inspection.

In this example,

$$\text{Recall} = 75\%,$$

which means the model detects 30 of the 40 real failures but misses 10. In other words, 25% of actual failures are not detected.

A false negative can lead to unexpected equipment breakdown, production downtime, repair costs, and potentially safety problems. A false positive generally results in an unnecessary inspection, which may be less costly than an undetected failure. For this reason, increasing recall would often be an appropriate priority.

Accuracy by itself can be misleading because the dataset is highly imbalanced. Only

$$\frac{40}{1000} \times 100 = 4\%$$

of the observations correspond to failures.

For example, a classifier that predicts "no failure" for every sample would obtain

$$\frac{960}{1000} = 96\%$$

accuracy while detecting zero actual failures.

Therefore, metrics such as recall, precision, and F1 score provide a more meaningful evaluation of the failure-detection system than accuracy alone.

One possible way to increase recall is to lower the classification threshold below 0.5. However, doing so would generally increase the number of false positives. The threshold should therefore be selected according to the relative cost of missed failures and unnecessary maintenance.

Summary of Final Answers

Question	Final Result
Q1	$f_t = 0.6248$, $i_t = 0.5939$, $\widetilde{C}_t = 0.4542$, $C_t = 0.5196$, $o_t = 0.6106$, $h_t = 0.2915$
Q2	5504 parameters with one bias per gate; 5632 with the PyTorch-style two-bias formulation.
Q3	$0.9^{10} = 0.3487$, $0.9^{50} \approx 0.0052$, $0.5^{50} \approx 8.88 \times 10^{-16}$, $0.99^{50} \approx 0.6050$.
Q4	26,280 records; 26,089 total windows; input $(64, 168, 3)$; output $(64, 24)$; 18,205 training windows; 285 batches; 17,408 LSTM parameters.
Q5	Normalised value = 0.75; prediction = failure; $TP = 30$, $FP = 15$, $FN = 10$, $TN = 945$; Precision = 0.6667; Recall = 0.7500; F1 = 0.7059; Accuracy = 97.5%.